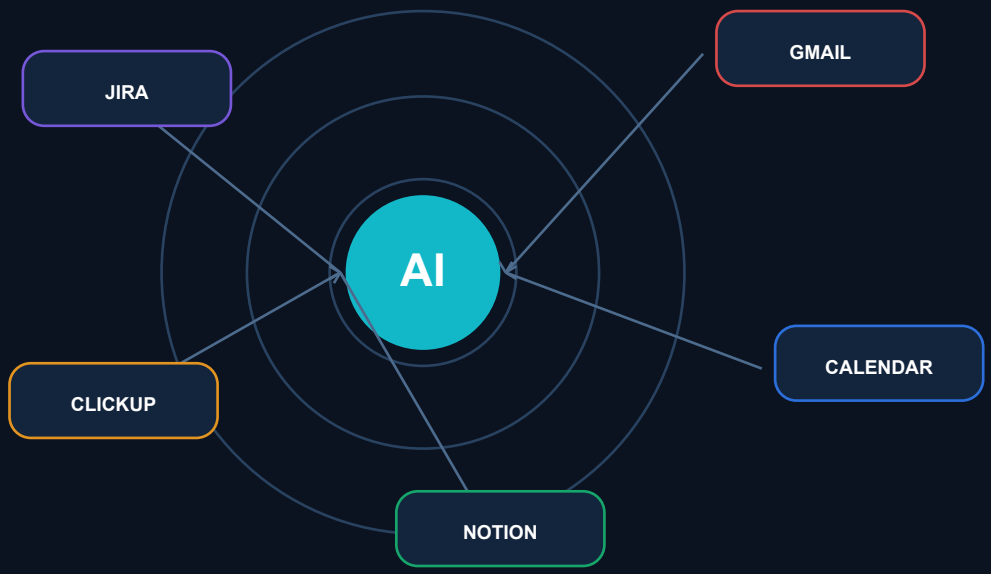
AI WORK OPERATING SYSTEM

Product and Architecture Guide

A detailed plan for turning Jira, ClickUp, Gmail, Google Calendar, Notion, and future workplace sources into one safe, context-aware assistant.



PRODUCT	ARCHITECTURE	DELIVERY
Vision, users, jobs, MVP	MCP + APIs, data, AI, security	Roadmap, backlog, risks, tests

Prepared for the hackathon-backend project | 18 July 2026

SECTION 00

How to use this guide

Read the first six pages for the product decision; use the remainder as an implementation reference.

If you are designing the product

Read pages 3-7. They define the promise, user journey, MVP boundary, and the central MCP-versus-API decision.

If you are designing the backend

Read pages 12-18. They define the context model, agent flow, sync strategy, tenancy, modules, and endpoints.

If you are implementing integrations

Read pages 8-11, then the source links on pages 24-25. Each provider has different maturity and compliance constraints.

If you are planning delivery

Read pages 19-23. They contain phases, acceptance criteria, cost, risks, testing, and the first four sprints.

Contents

Page	Topic
03	Executive assessment
04	Product promise
05	Users and jobs
06	Target architecture
07	Hybrid integration decision
08-11	Provider deep dives
12	Canonical context model
13	AI orchestration
14	Sync and retrieval
15	Security and tenancy
16	Current codebase assessment
17-18	Target backend and API
19	Roadmap
20	Testing and observability
21	Cost and vendor maturity
22	Risk register
23	First four sprints
24-25	Sources and glossary

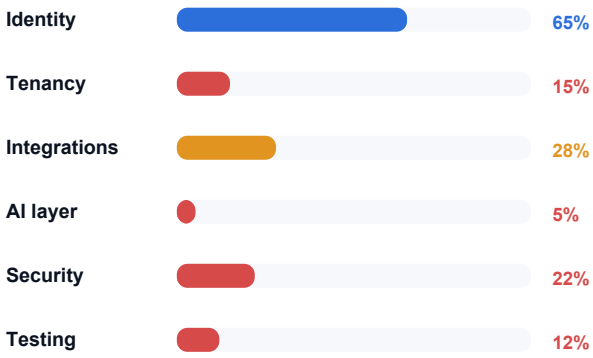
Scope note: this is an architecture assessment based on the repository snapshot and official vendor documentation available on 18 July 2026. Vendor previews, quotas, and pricing can change.

SECTION 01

Executive assessment

The idea is strong; the implementation is a foundation, not yet a product.

Recommendation: build one trustworthy daily-work loop before building many integrations. A user should connect accounts, receive a source-linked brief, ask a follow-up, and safely approve any write action.



Current repository: authentication, Prisma/PostgreSQL, user CRUD, Swagger, and Notion OAuth are present.

Missing product core: per-user integration ownership, actual provider reads, canonical work items, agent orchestration, background sync, citations, approvals, and production-grade tests.

Critical sequencing: fix tenancy and token safety before adding more providers.

Architecture decisions to make now

Decision	Recommended choice	Reason
Integration mode	Hybrid MCP + REST/webhooks	MCP is fast for live agent reads; APIs are deterministic for sync and scheduled briefs.
Tenancy	Every row belongs to user and organization	The current global connections table will leak context across users.
AI interface	Structured outputs with evidence links	Product trust depends on verifiable source items, not fluent prose alone.
Writes	Draft then explicit approval	External side effects must be inspectable, auditable, and reversible where possible.

SECTION 02

The product to build

One assistant that turns fragmented work signals into an evidence-backed plan.



The core user promise

- Tell me what matters now across tasks, email, documents, meetings, and conversations.
- Explain why an item is urgent and link me back to the original Jira issue, email thread, event, or document.
- Let me ask follow-up questions without repeating context.
- Draft actions for me, but do not send or mutate external systems until I approve.

The smallest compelling demo

Morning

The user opens a dashboard and sees three priorities, two meetings, one blocker, and one follow-up. Every claim has a source link.

Ask

The user asks: Why is the launch blocked? The assistant searches Jira and Notion, then returns evidence and uncertainty.

Prepare

Before a meeting, the assistant combines the calendar event, related tasks, recent notes, and open decisions into a briefing.

Act

The assistant drafts a Jira comment or email reply. The user reviews the target, content, and effect before approval.

North-star behavior: reduce the time from 'open workday' to 'confident next action' while preserving user control and source traceability.

SECTION 03

Users, jobs, and product boundaries

Design around decisions, not around a list of connected apps.

Persona	Primary job	First valuable output	Trust requirement
Individual contributor	Know priorities and blockers	Daily brief + focused follow-ups	Show source and freshness
Team lead	Understand delivery risk	Cross-project risk summary	Separate fact from inference
Manager	Prepare meetings and follow-ups	Meeting packet + commitments	Respect private messages
Operations/admin	Control connections and policy	Audit view + provider health	Least privilege and revocation

MVP jobs to be done

Daily brief

Summarize upcoming events, assigned work, urgent messages, blockers, and overdue follow-ups.

Ask across work

Answer a bounded question using authorized provider evidence and return direct source URLs.

Meeting preparation

Combine the event, participants, linked work, recent notes, and unresolved decisions.

Connection control

Connect, inspect, reauthorize, and disconnect each provider per user.

Explicitly out of scope for the first release

- Autonomous external writes, organization-wide knowledge ingestion, and a large multi-agent hierarchy.
- Every provider named in the vision document. Start with two high-value sources and one calendar source.
- A vector database as the primary truth store. Source records and provider IDs remain authoritative.
- Continuous full-mailbox ingestion. Gmail requires a narrow, privacy-first design and production verification.

SECTION 04

Recommended target architecture

A layered system that keeps provider access, context, reasoning, and side effects separate.



Key rule: the model never receives provider credentials and never bypasses the policy layer. It receives bounded tools whose handlers enforce user, organization, scope, freshness, and approval requirements.

Why this structure works

- Provider-specific complexity stays at the integration edge.
- The context platform can answer common questions even when one provider is temporarily unavailable.
- The AI layer reasons over normalized evidence and structured tools instead of raw credentials.
- Policy and audit controls remain deterministic application code.

SECTION 05

The central decision: MCP plus APIs

Do not force one mechanism to solve two different jobs.



Decision matrix

Need	MCP	REST API / webhook	Recommendation
Live user question	Excellent	Possible but more orchestration	MCP first
Scheduled daily brief	Fragile for bulk/background retrieval	Deterministic and retryable	API + queue
Incremental sync	Usually not the main abstraction	Cursors, history, sync tokens, webhooks	API + webhook
Agent-directed search	Tool descriptions fit model planning	Wrap endpoints as custom tools	Use MCP where mature
Audit and replay	Log calls but provider behavior can vary	Store request/cursor/event IDs	Persist application events
Writes	Convenient but high impact	Deterministic endpoint handlers	Approval-gated wrapper

Provider recommendation

Provider	MCP maturity	MVP choice
Jira	Official Atlassian Rovo MCP	Use MCP for live JQL/search; REST for scheduled sync.
ClickUp	Official public beta	Prototype with MCP; keep REST adapter ready for scale.
Gmail	Google Developer Preview	Use REST API in controlled pilot; defer public rollout.
Calendar	Google Developer Preview	REST events.list is the reliable MVP path.

SECTION 06A

Jira integration assessment

Best first task system: mature REST API plus an official production-oriented MCP surface.

What to read

Issues, projects, comments, status, priority, assignee, reporter, due dates, labels, links, worklogs, and custom fields.

Best queries

My unresolved work; overdue issues; blockers; recently changed issues; issues related to a meeting or document.

MCP

Rovo MCP supports issue retrieval, JQL search, project/metadata queries, comments, transitions, and cross-Atlassian search.

REST

Jira Cloud REST v3 and JQL are the deterministic path for sync, pagination, backfills, and scheduled briefs.

Recommended read flow

Step	Operation	Store
1	OAuth 2.0 3LO or Rovo MCP OAuth 2.1	Connection, site/cloud ID, granted scopes
2	JQL: assignee=currentUser() AND resolution=Unresolved	Issue IDs and updated timestamps
3	Fetch selected issue fields and comments	Normalized WorkItem + evidence URL
4	Sync by updated timestamp or webhook	Cursor, idempotency key, deletion state

Assessment

- Strength: excellent task semantics and strong source links; JQL makes bounded retrieval efficient.
- Risk: custom fields differ by site; never hard-code field IDs globally.
- Policy: all reads must inherit Jira permissions; writes require target preview and explicit confirmation.
- Cost: Atlassian says all Cloud customers can access Rovo MCP, with plan-based rate limits; Jira Free supports small teams.

Priority: HIGH. Jira plus Notion gives the MVP both execution state and project context.

SECTION 06B

ClickUp integration assessment

Broad work-management coverage, but the official MCP server is still public beta.

What to read

Tasks, Lists, Folders, Spaces, comments, members, Docs, custom fields, time entries, time-in-status, and chat channels.

MCP

Workspace search, task retrieval, comments, hierarchy, Docs, people, and time tools. Available on all plans with lower free limits.

REST

API v2 supports stable task and hierarchy reads, OAuth, webhooks, and plan-dependent rate limits.

Caveat

Workspace schemas and custom fields vary. Some administrative endpoints are Enterprise-only.

Recommended read flow

Step	Operation	Design note
1	OAuth 2.0 and authorized Workspaces	One user may authorize multiple Workspaces.
2	Read hierarchy: Workspace -> Space -> Folder -> List	Persist provider IDs and parent links.
3	Fetch tasks, comments, and selected custom fields	Normalize status without discarding native value.
4	Register signed webhooks	Deduplicate events and verify webhook signatures.

Assessment

- Strength: one provider can supply tasks, Docs, chat, and time evidence.
- Risk: beta MCP behavior and rate limits can change; isolate it behind a provider adapter.
- Free MCP without Everything AI is limited to 50 calls per rolling 24 hours; normal Free API access is 100 requests per minute per token.
- Do not implement Jira and ClickUp first unless the target users actually use both. They overlap heavily.

Priority: MEDIUM. Choose ClickUp instead of Jira when your demo organization lives in ClickUp; otherwise add it after the first task-system adapter is proven.

SECTION 06C

Gmail integration assessment

High product value and the highest compliance burden in this group.

What to read

Threads, messages, subject, sender, recipients, date, snippet, body, labels, attachments, and mailbox history.

MCP preview

search_threads finds conversations; get_thread retrieves headers and optional full bodies. Developer Preview limits public use.

REST

messages.list and threads.list search; messages.get and threads.get retrieve content; history.list supports incremental change processing.

Compliance

gmail.readonly is restricted. Public server-side storage or transmission can trigger verification and annual CASA assessment.

Privacy-first product design

Rule	Implementation
Retrieve narrowly	Use query windows such as is:unread newer_than:7d and fetch bodies only when needed.
Minimize retention	Store source IDs, headers, derived signals, and short-lived encrypted content where possible.
Protect prompts	Redact secrets and unnecessary personal data before model calls; disable sensitive trace capture.
Expose deletion	Provide user-visible disconnect, purge, and retention controls.
Separate pilot	Use test users or internal-only mode before a public production OAuth review.

Cost and maturity

- Standard Gmail API usage is currently available at no additional cost below the daily billing threshold.
- For newer projects, the documented threshold is 80,000,000 quota units per project per day; Google plans future charges above it later in 2026.
- The real early cost is engineering, privacy controls, verification time, and potentially an independent annual security assessment.

Priority: LATER PILOT. Build Calendar and a task source first; add Gmail when the data-handling and OAuth review plan is owned.

SECTION 06D

Google Calendar integration assessment

The simplest Google source and a strong anchor for daily briefs and meeting preparation.

What to read

Calendars, events, attendees, organizer, start/end, timezone, recurrence, location, Meet link, attachments, and availability.

MCP preview

List, search, and get events. Useful for interactive questions but still part of Google Workspace Developer Preview.

REST

events.list is stable, supports date windows, pagination, recurring-event expansion, update filters, and sync tokens.

Scope

calendar.events.readonly is usually sufficient for the product's read-first phase.

Recommended meeting-context flow

Stage	Action	Output
Discover	List primary and selected calendars	Authorized calendar IDs and access roles
Window	Fetch events from now through the next 7 days	Normalized Meeting work items
Enrich	Match title, attendees, attachments, and source links	Related Jira/ClickUp/Notion evidence
Prepare	Generate brief shortly before event	Agenda, open decisions, risks, and source links
Sync	Use nextSyncToken and handle 410 resets	Incremental and recoverable synchronization

Assessment

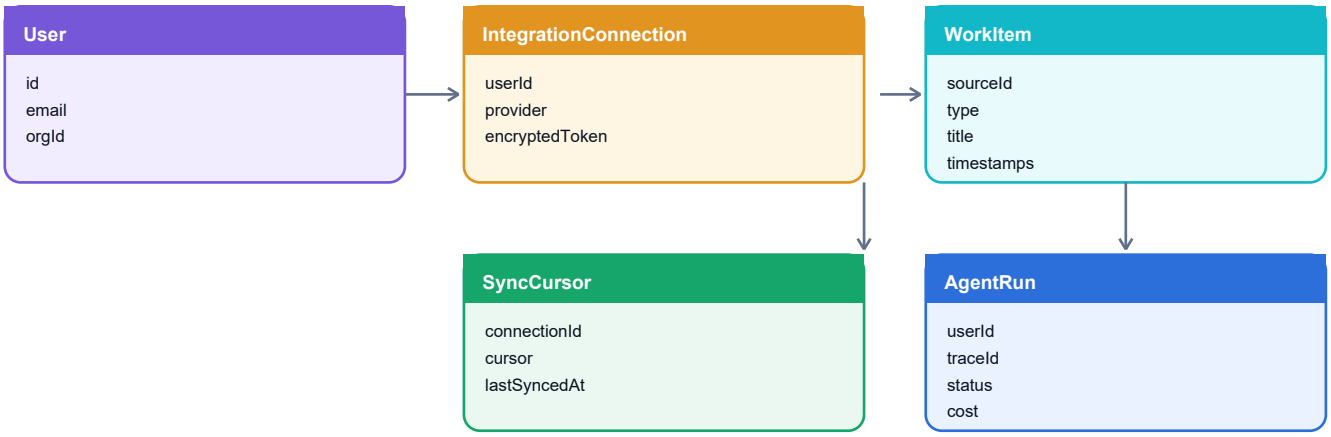
- Strength: bounded time ranges make retrieval cheap and explainable.
- Risk: private events can hide details depending on access role; never infer hidden content.
- New projects have a documented 1,000,000-request daily threshold before future billing; a hackathon is far below it.
- Meeting preparation becomes significantly better after adding a task and document source.

Priority: HIGH. Calendar is a low-complexity way to make the daily brief feel timely and personal.

SECTION 07

Canonical context model

Normalize enough to reason across providers, while preserving native source data and links.



Minimum WorkItem shape

Field group	Fields	Why
Identity	id, orgId, userId, provider, connectionId, sourceId	Tenant isolation and deduplication
Meaning	type, title, summary, status, priority	Cross-provider ranking and display
People	author, assignees, attendees	Ownership and meeting context
Time	createdAt, updatedAt, dueAt, startAt, endAt	Urgency, freshness, and sequencing
Evidence	sourceUrl, sourceVersion, excerpt, metadata	Traceable answers without losing fidelity
Lifecycle	syncedAt, deletedAt, contentHash, visibility	Incremental sync, deletion, and policy

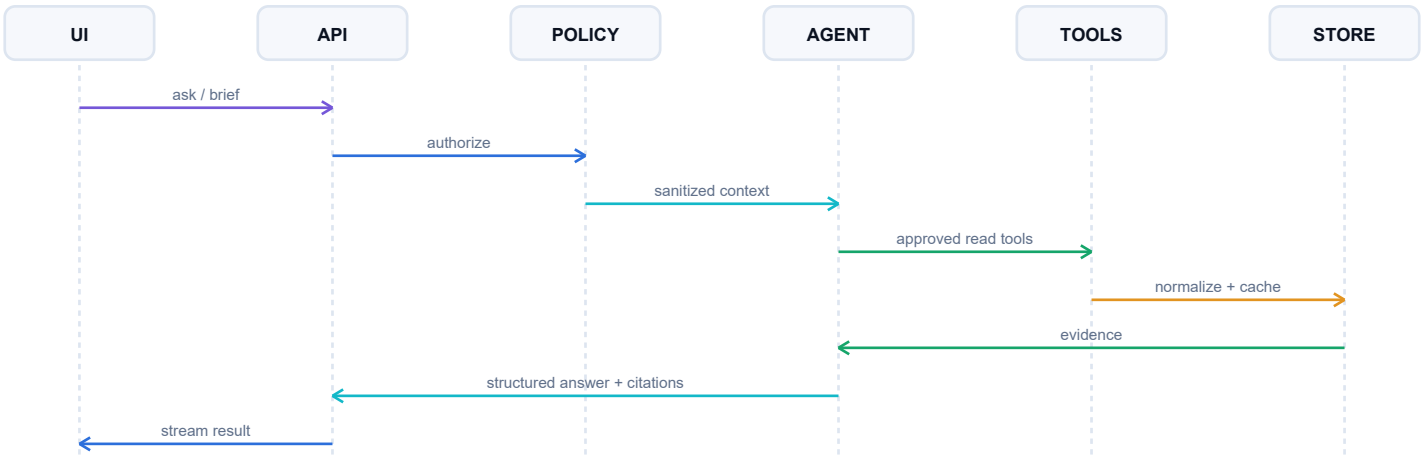
Storage layers

- PostgreSQL is the source of truth for users, connections, cursors, normalized records, approvals, and audit events.
- Object storage is optional for large, encrypted attachments; do not store them by default.
- Full-text search should precede vector retrieval. Add embeddings only for semantic queries that keyword filters cannot solve.
- Provider raw payloads should be encrypted, retention-limited, and excluded when normalized fields are sufficient.

SECTION 08

AI orchestration and evidence flow

The agent plans retrieval; deterministic application code owns authorization, tools, and side effects.



Recommended OpenAI layer

- Use the Responses API with remote MCP tools and application-defined function tools, or the TypeScript Agents SDK when you want its agent loop, sessions, approvals, guardrails, and tracing.

- Represent every provider result as structured evidence with provider, source ID, title, URL, timestamp, and access context.

- Require structured final output for briefs: summary, priorities, blockers, meetings, followUps, evidence, and uncertainty.

- Keep the first release to one orchestrator agent. Add specialist agents only after evals show the single-agent toolset is unreliable or too large.

Tool policy

Tool class	Default	Required controls
Read/search	Auto within granted scope	Tenant check, allowlist, timeout, result limit, audit
Expensive/broad read	Ask or narrow	Explain scope, cap pages, redact output
Draft	Allowed	No external effect; mark as draft
Write/delete/send	Approval required	Target preview, idempotency key, audit, result receipt

Trust rule: a useful answer without evidence is incomplete. A confident answer built from stale or unauthorized context is a defect.

SECTION 09

Synchronization, retrieval, and freshness

Use events when available, cursors when reliable, and scheduled reconciliation as the safety net.

Provider	Incremental mechanism	Fallback reconciliation	MVP cadence
Jira	Webhooks and updated-time JQL	Re-query recent updated window	5-15 min or on demand
ClickUp	Signed webhooks	List recently updated tasks	5-15 min or on demand
Gmail	history.list after watch notification	Bounded query by date/label	Pilot: on demand
Calendar	nextSyncToken	Full bounded window after 410 reset	15 min + before brief
Notion	Provider search/fetch or webhooks where suitable	Recent-page scan	On demand first

Queue design

Ingest

Receive webhook or schedule event; validate signature; write an immutable provider event.

Resolve

Load the connection and current cursor; reject disabled or mismatched tenant state.

Fetch

Call provider with pagination, timeout, retry, and rate-limit backoff.

Normalize

Upsert by provider + connection + sourceId; compute content hash and evidence metadata.

Index

Update full-text and optional semantic index after the database transaction commits.

Observe

Record provider latency, cursor lag, item counts, errors, and dead-letter state.

Correctness rules

- At-least-once delivery plus idempotent consumers is simpler and safer than pretending events arrive exactly once.
- Never advance a sync cursor until every page in the batch is committed.
- Keep provider deletion/tombstone semantics; do not silently resurrect deleted source records.
- Expose freshness in the UI: Live, synced N minutes ago, stale, or provider unavailable.

SECTION 10

Security, privacy, and multi-tenancy

These controls are architecture, not cleanup work.

Threat	Required control	Acceptance test
Cross-user data leak	userId + orgId on connections and work items; scoped repository methods	User A cannot list, fetch, ask about, or delete User B data
Token theft	Envelope encryption or managed secret vault; never return tokens	Database dump alone cannot reveal provider tokens
OAuth CSRF/account mix-up	State bound to user, provider, redirect, and expiry; PKCE where supported	Callback cannot attach a connection to another session
Prompt injection in source content	Treat provider content as untrusted data; tool allowlists and output filtering	A document cannot cause an unapproved send/delete
Over-broad ingestion	Minimum scopes, bounded queries, per-source enablement	Disconnect and purge are observable and complete
Sensitive traces	Disable or redact model/tool payload capture	Secrets and full email bodies are absent from logs

Authorization model

Identity

JWT subject resolves to an active application user and organization membership.

Connection

Every provider connection is owned by a user, optionally shared under explicit organization policy.

Tool

Each tool declares read/write risk, required scopes, provider, and approval mode.

Record

Repository queries always include tenant identifiers; source permissions are preserved.

Immediate repository fix: the current global IntegrationConnection model and unguarded /users and /integrations routes are incompatible with a multi-user product.

SECTION 11

Current codebase assessment

What exists, what is risky, and what can be retained.

Area	Current state	Assessment	Next move
Platform	NestJS 11, Prisma, PostgreSQL, validation, Swagger	Good foundation	Keep and modularize
Authentication	Signup/login, bcrypt, JWT, /auth/me	Useful but narrow	Add refresh/session policy and account lifecycle
Users	Public CRUD returning raw Prisma User	Critical exposure	Guard/admin-scope routes and map safe DTOs
Connections	Global rows; public list/get/delete	Critical tenant flaw	Add userId/orgId and guards
Notion OAuth	State, token exchange, persistence, revoke	Promising spike	Bind state/connection to user; encrypt token
Notion data	Hello endpoint only	Not implemented	Add search/fetch adapter
AI	No model or agent layer	Not implemented	Add structured brief and ask workflows
Testing	Generated hello tests	Insufficient	Service, authz, OAuth, sync, and e2e tests

What to preserve

- NestJS module boundaries, global validation, Swagger setup, Prisma service, password hashing, generic login errors, and single-use expiring OAuth state.
- Safe connection response projection that excludes accessToken and raw data.
- Provider-specific OAuth service as a starting pattern, after user binding and secret storage are corrected.

What to stop before adding providers

- Do not copy the global Notion connection schema into Jira, ClickUp, or Google.
- Do not expose Prisma entities directly from controllers.
- Do not store access tokens twice, including inside a raw OAuth payload.

Build verification note: repository dependencies were not installed during assessment, so the code review is static. Install dependencies and run build, lint, unit, and e2e checks before implementation begins.

SECTION 12

Target NestJS backend structure

Organize by capabilities and contracts, not by vendor endpoints alone.

Module	Responsibility	Key services
identity	Users, organizations, memberships, sessions	IdentityService, SessionService
connections	OAuth state, connection lifecycle, encrypted credentials	ConnectionService, TokenVault
providers	Provider adapters behind one contract	JiraAdapter, ClickUpAdapter, GmailAdapter, CalendarAdapter
sync	Webhooks, schedules, cursors, jobs, retries	SyncCoordinator, ProviderEventConsumer
context	Normalization, evidence, search, ranking	WorkItemService, EvidenceService, SearchService
assistant	Briefs, ask workflow, prompts, structured output	BriefService, AskService, ToolRegistry
approvals	Drafts, pending actions, execution receipts	ApprovalService, ActionExecutor
audit	Security and product events	AuditService, MetricsService

Provider adapter contract

Method	Purpose
authorize / callback / revoke	Lifecycle and credentials
listCapabilities	Declare tools, scopes, preview status, and write risk
search	Bounded live query returning Evidence[]
sync	Cursor-based page retrieval returning normalized upserts/tombstones
execute	Approval-gated write with idempotency and receipt
health	Connection status, last success, scope mismatch, and reauthorization

Keep provider adapters replaceable. An official MCP beta may become the best path later, while a REST adapter remains the reliable fallback today.

SECTION 13

Proposed product API surface

A small set of user-facing capabilities can hide substantial provider complexity.

Method	Endpoint	Purpose	Auth / approval
GET	/connections	List current user's providers and health	JWT
POST	/connections/:provider/start	Begin provider OAuth	JWT
GET	/connections/:provider/callback	Complete bound OAuth state	State + session
DELETE	/connections/:id	Revoke and purge connection	Owner JWT
POST	/assistant/daily-brief	Generate structured source-linked brief	JWT
POST	/assistant/ask	Ask across authorized workplace context	JWT
GET	/assistant/runs/:id	Read status, evidence, and result	Owner JWT
POST	/actions/draft	Prepare a write action without executing	JWT
POST	/actions/:id/approve	Execute exactly the previewed action	Owner JWT + approval
POST	/webhooks/:provider	Receive signed provider events	Provider signature

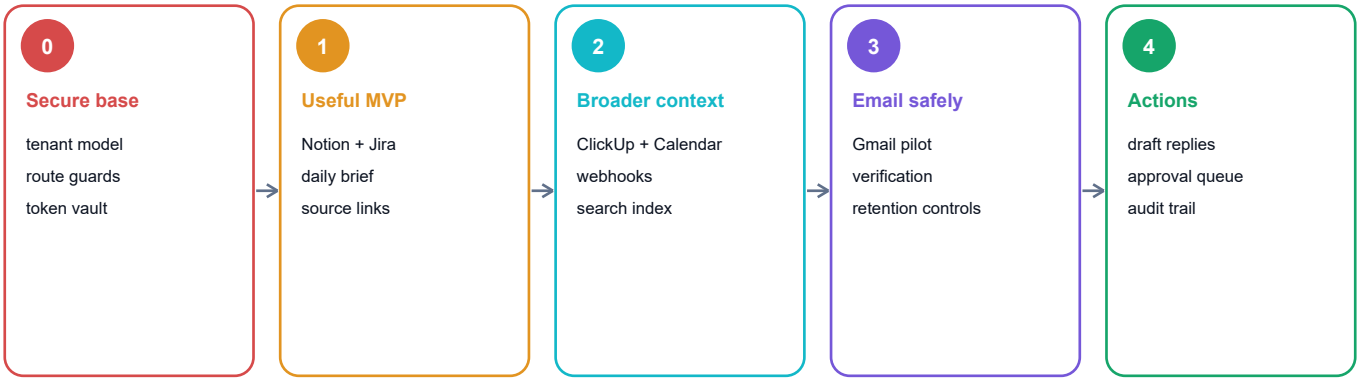
Daily brief response contract

Section	Required content
summary	Two to four sentences, with freshness statement
priorities	Rank, reason, urgency, owner, due time, source evidence
meetings	Time, purpose, preparation items, related work, links
blockers	Known fact, inferred risk, affected items, recommended next step
followUps	Person, commitment, age, source, suggested draft
evidence	Provider, sourceId, title, URL, updatedAt, excerpt

SECTION 14

Delivery roadmap

Each phase should deliver a user-visible improvement and retire a major risk.



Phase gates

Phase	Exit criteria
0 - Secure base	No public user/connection exposure; per-user connection ownership; encrypted tokens; authz e2e tests.
1 - Useful MVP	One task source + Notion + daily brief; every claim linked to evidence; manual refresh works.
2 - Broader context	Calendar and second provider adapter; queue, cursors, retries, health, and freshness visible.
3 - Email safely	Gmail pilot with minimum scopes, retention controls, redaction, and verification path documented.
4 - Controlled actions	Draft/approve/execute flow; audit receipt; negative tests block unapproved writes.

MVP acceptance criteria

- A new user can connect providers without affecting another user's data.
- A daily brief completes within the agreed latency budget and labels stale/unavailable sources.
- Every priority and blocker has at least one source link or is explicitly marked as inference.
- Disconnect removes active access and starts the documented purge workflow.
- No external write can execute without a persisted approval tied to the exact action payload.

SECTION 15

Testing, evaluation, and observability

A trustworthy assistant needs software tests and answer-quality evals.

Layer	Tests / signals	Failure caught
Unit	DTOs, policy, ranking, normalization, cursor logic	Local logic regression
Contract	Recorded provider fixtures and schema drift tests	Vendor response change
Integration	Postgres, encryption, queue, OAuth callback	Boundary failure
E2E authz	Two users, two orgs, hostile IDs and callbacks	Cross-tenant data leak
Agent eval	Golden questions, expected evidence, hallucination checks	Fluent but unsupported answer
Safety eval	Prompt injection, broad reads, write attempts	Policy bypass
Operations	Sync lag, 429s, provider errors, token expiry, DLQ	Silent staleness

Metrics that matter

Product

Brief opened, priority clicked, follow-up completed, time-to-first-useful-action.

Quality

Evidence coverage, unsupported-claim rate, stale-source rate, user correction rate.

Reliability

Provider success, sync lag, job retries, dead letters, OAuth reauthorization rate.

Cost

Model tokens per workflow, tool calls, provider requests, cache hit rate, cost per active user.

Tracing caution

- The OpenAI Agents SDK can trace model generations, tool calls, guardrails, and handoffs, which is valuable for debugging.
- Because traces can contain sensitive tool inputs and outputs, disable sensitive-data capture or redact provider content before export.
- Store application audit events separately from model traces; audit records must be deterministic and retention-controlled.

SECTION 16

Cost, availability, and vendor maturity

For the hackathon, platform access is mostly free; production compliance and engineering dominate early cost.

Provider	Prototype cost	Limits / maturity	Production concern
Jira	Free plan available; MCP available to Cloud customers	Free MCP 500 calls/hour	Jira plan, admin policy, site field variance
ClickUp	API and MCP available on Free	Free MCP 50 calls/24h; MCP public beta	Higher plan/add-on for more capacity
Gmail	Standard API use free below threshold	MCP Developer Preview; restricted scopes	OAuth verification and possible annual assessment
Calendar	API free below threshold	MCP Developer Preview	Future over-threshold billing; Workspace preview rules
OpenAI	Usage-based model/tool cost	Depends on model, context, and tool calls	Budget caps, caching, eval-driven routing

Documented Google thresholds for new projects

API	No-extra-charge daily threshold	Status
Gmail	80,000,000 quota units / project / day	Future charges above threshold planned later in 2026; details pending
Calendar	1,000,000 requests / project / day	Future charges above threshold planned later in 2026; details pending

Cost controls to design now

- Bound queries by time, assignee, label, project, and page count before calling a model.
- Cache normalized evidence and brief components; do not repeatedly retrieve unchanged content.
- Track cost per workflow and per active user, not only monthly total.
- Use smaller/cheaper model paths for classification and ranking only after evals prove acceptable quality.

Financial assessment: the hackathon can run cheaply. The production budget risk is Gmail compliance plus engineering for safe multi-tenant synchronization, not basic provider API fees.

SECTION 17

Risk register and decision log

Resolve high-impact architecture risks before expanding feature count.

Risk	Likelihood	Impact	Mitigation	Owner decision
Cross-tenant leak	High today	Critical	Schema ownership, guards, scoped repositories, e2e isolation	Block feature work until fixed
Provider preview changes	Medium	High	Adapter contract, REST fallback, capability flags	Do not couple domain to MCP names
Gmail review delay	High	High	Internal pilot, minimal scopes, Calendar first	Defer public Gmail launch
Prompt injection	High	High	Untrusted-content boundary, tool policy, approvals	No raw provider instructions
Stale answers	Medium	High	Freshness metadata, live fallback, visible status	Evidence includes updatedAt
Rate limits	Medium	Medium	Queue, backoff, cache, bounded retrieval	Per-provider budgets
Overbuilding agents	Medium	Medium	Single orchestrator, eval-driven specialization	No multi-agent MVP
Schema drift	Medium	Medium	Contract fixtures and unknown-field tolerance	Alert, do not crash ingest

Open decisions for the team

- Which real users and workspace will the demo use: Jira or ClickUp? Choose one as the first task source.
- Will organization admins ever share a connection, or is every connection strictly personal in the first release?
- What content may be retained, for how long, and which sources are live-read only?
- What is the initial write boundary: no writes, drafts only, or one approval-gated provider action?
- What freshness and latency targets define a successful daily brief?

Architecture is ready to begin once the first task provider, tenant model, and retention boundary are decided. Everything else can evolve behind contracts.

SECTION 18

First four sprints

A practical sequence from the current repository to a demonstrable product loop.

Sprint	Build	Demo outcome	Definition of done
1 - Secure foundation	User/org ownership; safe DTOs; guards; token vault; migration	Two users cannot see or mutate each other's data	Authz e2e tests + migration + secrets excluded
2 - First context	Complete Notion reads; add Jira or ClickUp adapter; WorkItem model	Search actual docs and assigned work	Evidence URLs, pagination, normalized fixtures
3 - Assistant loop	Daily brief + ask endpoint; structured output; tool registry; citations	Source-linked morning brief and follow-up question	Golden eval set and unsupported-claim checks
4 - Time + reliability	Calendar; queue; cursors; retries; health; freshness UI contract	Meeting-aware brief that survives provider errors	Sync metrics, DLQ path, stale labels, recovery test

Sprint 1 concrete backlog

- Add Organization, Membership, and user-owned IntegrationConnection relations in Prisma.
- Bind OAuthState to userId, provider, redirect target, code verifier where supported, and consumedAt.
- Encrypt provider tokens once; remove access_token from stored raw payloads.
- Guard /users and /integrations; replace Prisma entities with safe response DTOs.
- Make /auth/me load only the current user's connections.
- Add two-user authorization e2e tests and OAuth callback account-mix-up tests.
- Install dependencies and establish build, lint, unit, and e2e checks in CI.

Suggested team split

Platform

Tenant schema, security, queue, context model, assistant API.

Provider owner

OAuth, adapter, fixtures, quotas, failure modes, handoff notes.

Product/UI

Connection health, daily brief, evidence links, freshness, approvals.

Quality

Cross-tenant tests, provider contracts, eval dataset, operational dashboard.

SECTION A

Official sources: providers

Use these as the source of truth before implementation; verify preview and quota details again at release time.

A1 - Atlassian Rovo MCP overview
https://developer.atlassian.com/cloud/rovo-mcp/

A2 - Atlassian Rovo MCP getting started
https://developer.atlassian.com/cloud/rovo-mcp/guides/getting-started/

A3 - Atlassian Rovo MCP supported tools
https://developer.atlassian.com/cloud/rovo-mcp/guides/supported-tools/

A4 - Atlassian Rovo MCP authentication
https://developer.atlassian.com/cloud/rovo-mcp/guides/authentication-and-authorization/

A5 - Atlassian MCP access and rate limits
https://www.atlassian.com/platform/rovo-mcp

A6 - Jira Cloud REST API v3 introduction
https://developer.atlassian.com/cloud/jira/platform/rest/v3/intro

A7 - Jira OAuth 2.0 3LO
https://developer.atlassian.com/cloud/jira/platform/oauth-2-3lo-apps/

A8 - Jira issue search and JQL
https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-issue-search/

C1 - ClickUp MCP setup and limits
https://developer.clickup.com/docs/connect-an-ai-assistant-to-clickups-mcp-server

C2 - ClickUp MCP supported tools
https://developer.clickup.com/docs/mcp-tools

C3 - ClickUp authentication
https://developer.clickup.com/docs/authentication

C4 - ClickUp API availability by plan
https://developer.clickup.com/docs/apis-available-by-plan

C5 - ClickUp API rate limits
https://developer.clickup.com/docs/rate-limits

C6 - ClickUp webhooks
https://developer.clickup.com/docs/webhooks

Repository sources

- PROJECT_DESCRIPTION.md for the AI Work OS vision and teammate integration ownership.

- prisma/schema.prisma for current User, IntegrationConnection, and OAuthState models.

- src/auth, src/users, and src/integrations for the current authentication and Notion implementation.

SECTION B

Official sources: Google and OpenAI

Google MCP features are Developer Preview as of this guide's publication date.

G1 - Google Workspace Developer Preview
https://developers.google.com/workspace/preview

G2 - Gmail MCP get_thread
https://developers.google.com/workspace/gmail/api/reference/mcp/tools_list/get_thread

G3 - Gmail API messages.list guide
https://developers.google.com/workspace/gmail/api/guides/list-messages

G4 - Gmail OAuth scopes
https://developers.google.com/workspace/gmail/api/auth/scopes

G5 - Gmail API quotas and pricing
https://developers.google.com/workspace/gmail/api/reference/quota

G6 - Calendar MCP list_events
https://developers.google.com/workspace/calendar/api/v3/reference/mcp/tools_list/list_events

G7 - Calendar events.list
https://developers.google.com/workspace/calendar/api/v3/reference/events/list

G8 - Calendar API quotas
https://developers.google.com/workspace/calendar/api/guides/quota

G9 - Google restricted-scope security assessment
https://support.google.com/cloud/answer/13465431

O1 - OpenAI MCP and connectors
https://developers.openai.com/api/docs/guides/tools-connectors-mcp

O2 - OpenAI Agents SDK TypeScript
https://openai.github.io/openai-agents-js/

O3 - OpenAI Agents SDK guardrails
https://openai.github.io/openai-agents-js/guides/guardrails/

O4 - OpenAI Agents SDK tracing
https://openai.github.io/openai-agents-js/guides/tracing/

Glossary

Term	Meaning in this design
MCP	A standard tool interface that lets an AI client call provider capabilities.
Adapter	Application boundary that hides provider-specific MCP or REST details.
WorkItem	Canonical record representing a task, message, document, meeting, or decision.
Evidence	Source-linked record used to support an assistant claim.
Cursor	Provider-specific checkpoint for incremental synchronization.
Guardrail	Input, output, or tool check that blocks or reshapes unsafe behavior.
Approval	Persisted user consent to execute one exact external action.